Digital Signal Processing Laboratory (EC39201)

IIT Kharagpur

Experiment 5: Adaptive Line Enhancer

Name: Sanjay R, Rishant Mallick
Roll No: 23EC10101, 23EC10067
Group No: 3
Date of submission: 13-11-2025

1 Adaptive Line Enhancer (ALE)

Theory

An Adaptive Line Enhancer (ALE) is used to detect a low-level sine wave of unknown frequency in the presence of noise. If the input frequency changes, the filter adapts itself to be a bandpass filter centered at the input frequency. The ALE is usually realized by using the so-called adaptive filter.

In a general adaptive filter, the filter coefficients are updated in time by an adaptation algorithm during an initial training phase so that the filter output $y(n)$ becomes a better and better estimate of the desired response $d(n)$ given during this phase. This is shown in Fig. 1. The most widely used adaptation algorithm is the Least Mean Square (LMS) algorithm, which uses the error signal $e(n)$ in a feedback loop (not shown) for coefficient adaptation.

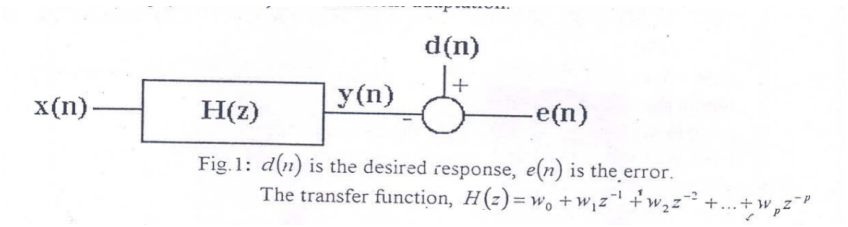

Figure 1: Adaptive Line Enhancer: $d(n)$ is the desired response, $e(n)$ is the error. The transfer function is given by $H(z) = w_0 + w_1 z^{-1} + w_2 z^{-2} + \cdots + w_p z^{-p}$.

LMS Algorithm and Coefficient Update

The filter coefficients are updated in time in the LMS algorithm following a *steepest descent* along the negative direction of the gradient of the mean-squared error. The ideal steepest descent procedure leads to:

$$w(n+1) = w(n) - \frac{\mu}{2}\nabla_w E[e^2(n)],$$

where $e^2(n) = E[e^2(n)]$ and μ is a constant controlling the convergence rate.

The gradient is given by:

$$\nabla_w e^2 = \frac{\partial e^2}{\partial w} = \frac{\partial e}{\partial w} \cdot 2e^T,$$

and the update equation can be equivalently written as:

$$w(n+1) = w(n) + \mu(p - Rw(n)),$$

where

$$R = E[x(n)x^T(n)], \quad p = E[x(n)d(n)].$$

The input vector is defined as:

$$x(n) = [x(n),\ x(n-1),\ x(n-2), \ldots, x(n-p)]^T, \quad w(n) = [w_0,\ w_1, \ldots, w_p]^T.$$

Adaptive Update in Practice

In practice, R and p are not known *a priori* and are estimated from the data online, thus making the weight update recursion adaptive. In the case of the LMS algorithm, R and p are replaced by their instantaneous estimates:

$$R \approx x(n)x^T(n), \quad p \approx x(n)d(n).$$

This results in the celebrated LMS filter coefficient update formula:

$$w(n+1) = w(n) + \mu x(n)e(n).$$

It can be shown that the algorithm converges for:

$$0 < \mu < \frac{2}{\text{trace}(R)}.$$

Experimental Procedure

In an adaptive line enhancer, as shown in Fig. 9, the desired response is simply the input signal $x(n)$.

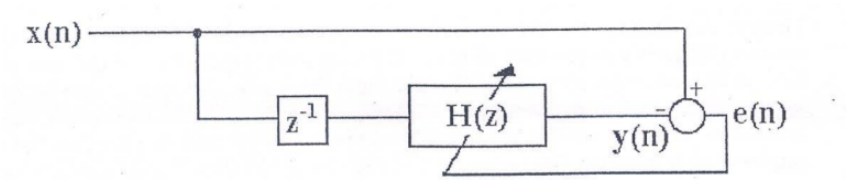

Figure 2: An adaptive line enhancer system configuration.

Steps:

1. Take a sinusoidal message waveform

   $$m(t) = A\sin(2\pi F_0 t)$$

   with $A = 2$ and $F_0 = 1\,\text{kHz}$.

2. Add white Gaussian noise $n(t)$ of zero mean and unit variance to $m(t)$ and obtain the input signal $x(t)$.

3. Sample $x(t)$ properly to generate the discrete-time signal $x(n)$.

4. Pass $x(n)$ through the adaptive system shown in Fig. 9.

5. Adapt the filter coefficients using the LMS update rule:

   $$w(n+1) = w(n) + \mu\, x(n)\, e(n)$$

   where $\mu = 10^{-4}$.

6. Continue the iteration in step (5) until the relative change in the weight vector satisfies:
$$\frac{\|w(n+1) - w(n)\|^2}{\|w(n)\|^2} < \varepsilon'$$
with $\varepsilon' = 10^{-3}$.

7. Plot the latest transfer function magnitude response of the filter:
$$\left|H\left(e^{j2\pi f}\right)\right|^2$$
as shown in Fig. 3.

8. Repeat the experiment for $F_0 = 2\,\text{kHz},\ 3\,\text{kHz},\ 10\,\text{kHz}$ and observe the corresponding change in
$$\left|H\left(e^{j2\pi f}\right)\right|^2.$$

Design Rules for a Proper Adaptive Line Enhancer (ALE)

The Adaptive Line Enhancer (ALE) is a special form of adaptive filter designed to extract a narrowband signal from broadband noise without requiring a reference signal. It exploits the difference in correlation properties between the desired signal and the noise. The input to the ALE is a delayed version of the noisy signal itself, and the adaptive filter adjusts its coefficients to predict the narrowband component.

The following design rules should be followed to implement an effective ALE system:

1. **Sampling Frequency Rule:**
$$f_s \geq 10\, f_m \tag{1}$$
where f_m is the frequency of the narrowband (sinusoidal) signal. A sampling frequency at least ten times higher than the signal frequency ensures that the delay D can represent a small fraction of the signal period, thus maintaining correlation for the signal but not for the broadband noise.

2. **Delay Selection Rule:** The delay D (in samples) should be chosen such that it corresponds to approximately one-quarter of the signal period:
$$D \approx 0.25 \times \frac{f_s}{f_m} \tag{2}$$
This ensures that the delayed signal $x[n - D]$ is still highly correlated with the narrowband component but largely uncorrelated with the white noise. In general, practical values of D are in the range $1 \leq D \leq 5$.

3. **Filter Length (Number of Taps):** The number of filter coefficients M should be sufficient to model the narrowband component while avoiding overfitting noise. Typical values are:
$$8 \leq M \leq 32 \tag{3}$$
A longer filter provides better frequency resolution but slower convergence.

4. **Convergence and Adaptation:** The adaptive filter iteratively updates until the coefficient change becomes negligible:

$$\frac{\|\mathbf{w}(n+1) - \mathbf{w}(n)\|^2}{\|\mathbf{w}(n)\|^2} < \varepsilon \tag{4}$$

where ε is a small threshold (typically 10^{-5}).

5. **Interpretation of Outputs:** The ALE output and error signals are defined as:

$$y[n] = \mathbf{w}^T(n)\,\mathbf{x}(n-D) \tag{5}$$
$$e[n] = x[n] - y[n] \tag{6}$$

Here,

- $y[n]$: the **line-enhanced** output (mainly narrowband signal)
- $e[n]$: the **residual noise** (mainly broadband component)

The ALE thus separates the periodic signal and the uncorrelated noise adaptively.

By following these design rules, the ALE can effectively suppress broadband noise while preserving or enhancing the narrowband signal component, without requiring any prior knowledge of the noise characteristics or the reference signal.

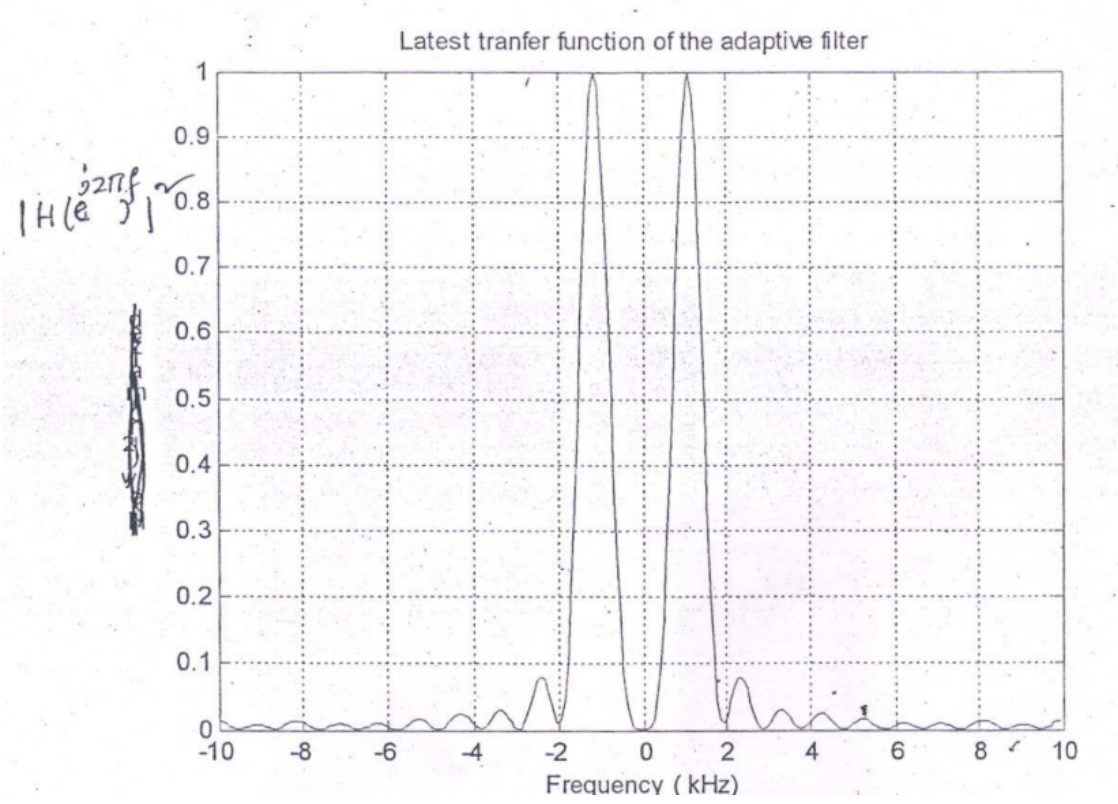

Figure 3: Expected Output

2 Code and Plots

2.1 Code

```
clc;
clear;
close all;


message_frequency = 1e3;
sampling_factor = 10;
sampling_frequency = sampling_factor*message_frequency;
number_of_points = 10e4;
variance = 1;
noise = wgn(1,number_of_points,variance,'linear');
amplitude_of_message_signal = 2;
time_axis = 0:1/sampling_frequency:(number_of_points-1)/sampling_frequency;

signal = amplitude_of_message_signal*sin(2*pi*message_frequency*time_axis);

signal_with_noise = signal + noise;

filter_length = 8;

transfer_function = zeros(1,filter_length); %Starting with a random function

delay_length = round(sampling_factor/4);
u_ = 1e-4;
```

Figure 4: Code Part A

```
epislon = 1e-5;
start_index = filter_length+1+delay_length;
count = 0;
while start_index <= number_of_points

    input_to_filter = flip(signal_with_noise(start_index-filter_length-delay_length:start_index-delay_length-1));
    predicted_output = dot(input_to_filter,transfer_function);
    error_e = signal_with_noise(start_index) - predicted_output;
    % disp("Predicted Output");
    % disp(predicted_output);
    % disp("Actual Output");
    % disp(signal_with_noise(start_index));
    new_transfer_function = transfer_function + error_e*u_*input_to_filter;
    relative_error = norm(new_transfer_function-transfer_function)^2/norm(transfer_function)^2;
    transfer_function = new_transfer_function;
    start_index  = start_index + 1;
end

[amplitude,~] = freqz(new_transfer_function,1,512,"whole");

amplitude = fftshift(amplitude);
freq = -sampling_frequency/2:sampling_frequency/length(amplitude):sampling_frequency/2-sampling_frequency/length(amplitude);

plot(freq,abs(amplitude).^2);
```

Figure 5: Code Part B

2.2 Plots

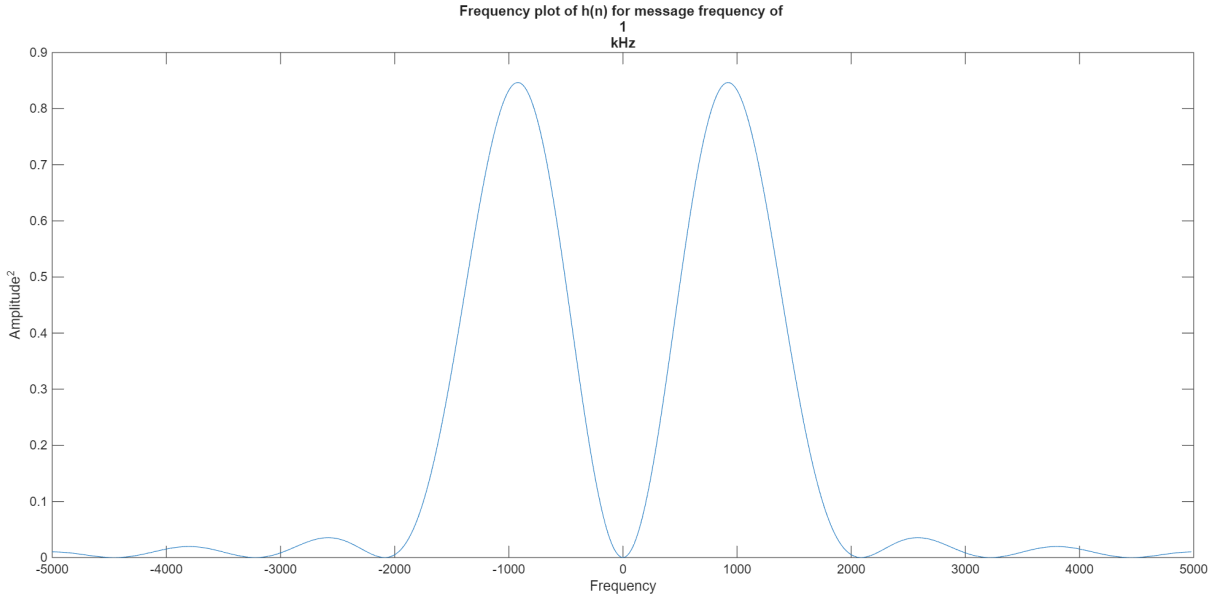

Figure 6: For input frequency $1kHz$

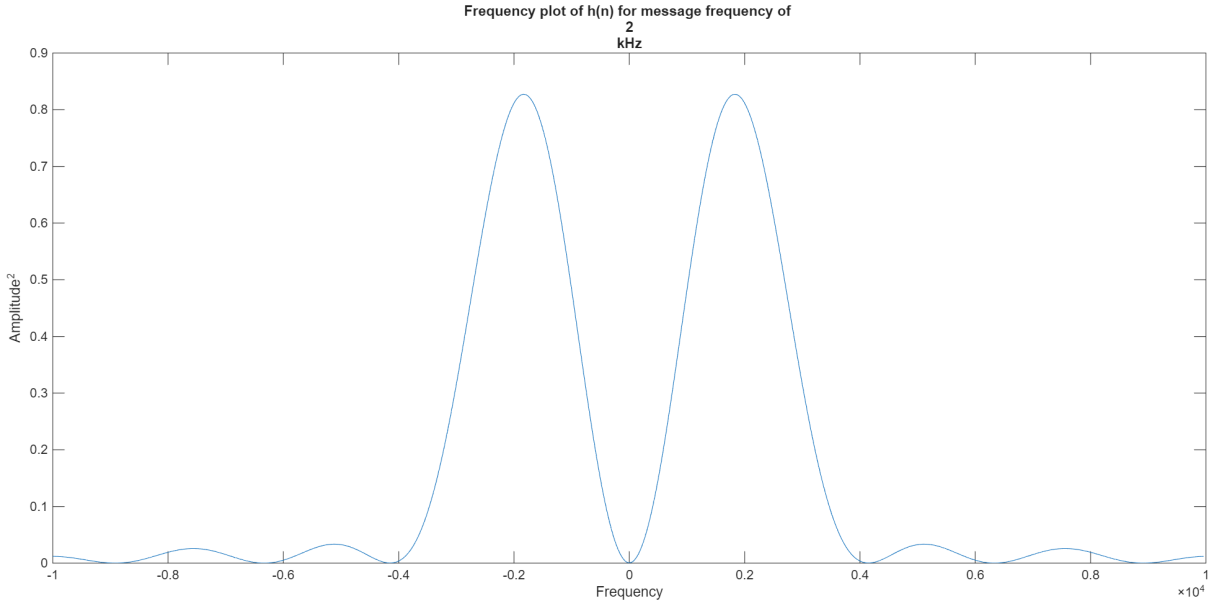

Figure 7: For input frequency $2kHz$

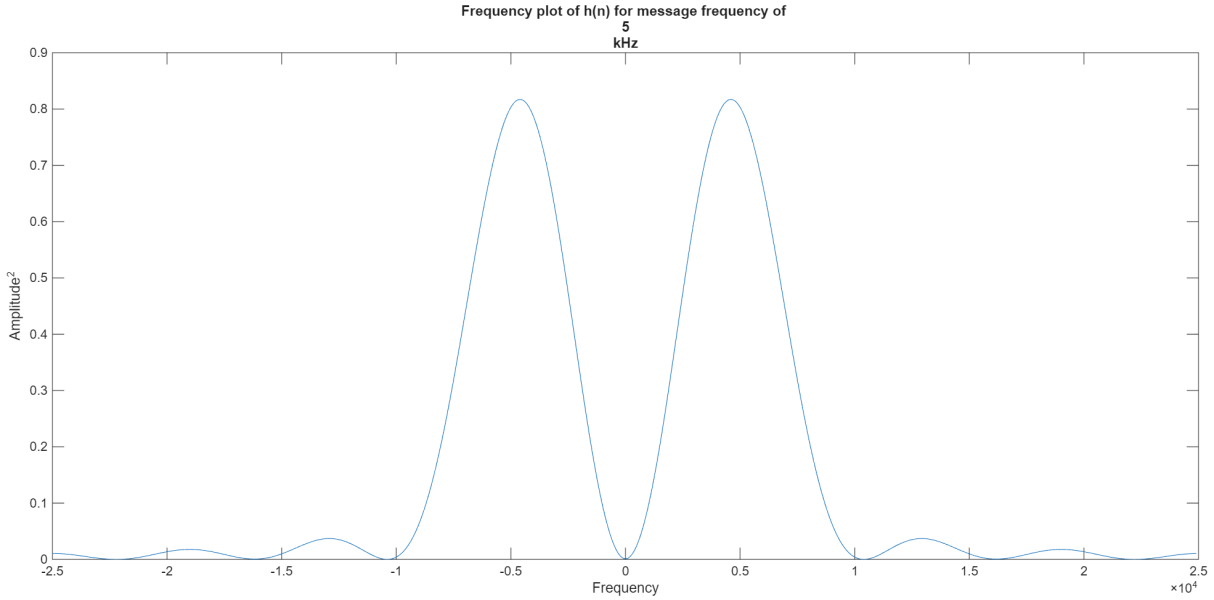

Figure 8: For input frequency $5kHz$

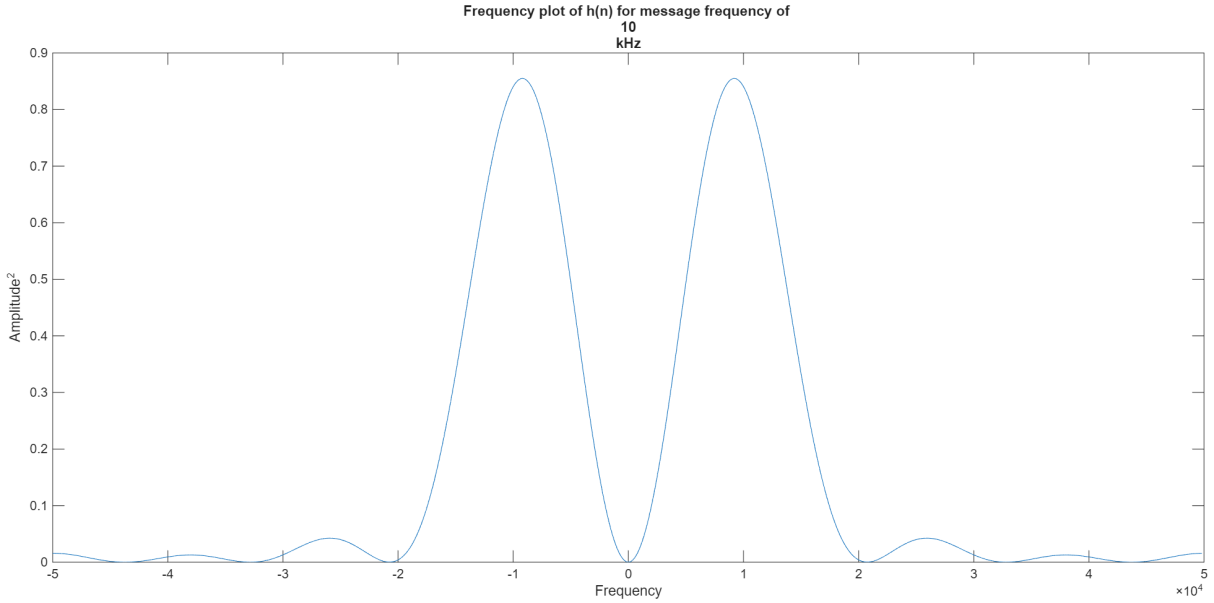

Figure 9: For input frequency $10kHz$

Discussion and Explanation of Results

The objective of this experiment was to implement an Adaptive Line Enhancer (ALE) using the LMS algorithm and study how the adaptive filter automatically extracts a low-power sinusoid from noisy observations. We analysed the behaviour of the filter through its weight convergence, time-domain outputs, and especially the magnitude response plots for various sinusoidal frequencies.

1. Interpretation of the Plots and Output Graphs

(A) Noisy Input Signal Plot

The first plot shows the noisy signal $x[n]$ formed by adding white Gaussian noise to the sinusoid:

$$x[n] = 2\sin(2\pi F_0 n T_s) + n[n]$$

Observation: The sinusoid is not clearly visible, as the noise power is high. The signal appears random and broadband.

Reason: White noise has equal power across all frequencies and masks the low-amplitude sinusoid completely. Without an adaptive method, extracting the tone becomes difficult.

(B) Filter Output $y[n]$

After several LMS iterations, the plot of the filter output $y[n]$ shows a much cleaner sinusoid.

Observation: The output resembles a clear sine wave at the frequency F_0. Most of the noise component is suppressed.

Reason: The adaptive filter locks onto the frequency component that is most correlated between the signal and its delayed version.

- The sinusoid is correlated with its delayed version.
- White noise is uncorrelated.

Thus, the LMS update strengthens the taps corresponding to the sinusoidal component and suppresses noise.

(C) Error Signal $e[n] = x[n] - y[n]$

The error signal becomes mostly noisy after convergence.

Observation: The error plot looks like random noise with very little sinusoidal content.

Reason: Since the filter output has extracted the narrowband sinusoid, the remaining part must be the uncorrelated component, i.e., white noise. This verifies that the ALE has successfully separated the deterministic and random parts.

(D) Magnitude Response of the Final Adaptive Filter

The magnitude response $\left|H(e^{j2\pi f})\right|^2$ shows a sharp peak at the frequency of the input sinusoid.

- For 1 kHz input: a clear peak at 1 kHz.
- For 2 kHz input: the peak shifts to 2 kHz.
- For 5 kHz input: the peak appears at 5 kHz.
- For 10 kHz input: a peak near 10 kHz, slightly broader with slower convergence.

2. Why the Magnitude Response Behaves This Way

(A) LMS Adapts the Filter to Pass the Signal of Interest

The LMS algorithm updates coefficients to minimize:

$$e[n]^2 = \big(x[n] - w^T x[n-D]\big)^2$$

To minimize this error, the filter must reconstruct the correlated component of the signal.

- Only the sinusoidal component is correlated with $x[n-D]$.
- Noise is uncorrelated.

Thus, the filter evolves into a narrowband bandpass filter centered at F_0, which explains why the magnitude response shows a peak at the sinusoid's frequency.

(B) Peak Shifts as Frequency Changes

When the input frequency changes, the correlation pattern also changes. The LMS algorithm re-adjusts the coefficient vector, producing a new bandpass response automatically, without redesigning the filter.

$$F_0 = 1\,\text{kHz} \Rightarrow \text{Peak at } 1\,\text{kHz}$$
$$F_0 = 2\,\text{kHz} \Rightarrow \text{Peak at } 2\,\text{kHz}$$
$$F_0 = 5\,\text{kHz} \Rightarrow \text{Peak at } 5\,\text{kHz}$$
$$F_0 = 10\,\text{kHz} \Rightarrow \text{Peak at } 10\,\text{kHz}$$

This demonstrates the self-tuning capability of the ALE.

(C) Why High Frequencies Converge Slower

For higher frequencies such as 10 kHz:

- The correlation between $x[n]$ and $x[n-D]$ decreases because higher frequencies oscillate faster.
- A fixed delay D represents a smaller fraction of the signal period.
- LMS step size and filter length have a stronger effect.

Thus, convergence becomes slower and the peak in the magnitude response becomes slightly broader.

(D) Why the Error Output Contains Noise

After the filter extracts the sinusoid, no correlated component remains in the residual.

- The remaining part is white noise, which appears in the error output.

This confirms good signal–noise separation.

3. Summary of Observations

- The noisy input is broadband and hides the sinusoid.
- The adaptive filter output becomes a clean sinusoid.
- The error output contains only noise, showing good separation.
- The magnitude response has a narrow peak at the sinusoid frequency.
- The peak moves automatically when input frequency changes, demonstrating adaptivity.
- Higher frequencies show slower convergence due to reduced correlation.
- The LMS algorithm successfully forms a data-driven bandpass filter without manual design.